# Horizon: correlated validation of the range relation

Algebraic supplement and experimental protocol

Technical dossier of the ZK-agentiK repository

17 October 2026 Experimental version 1

**Abstract**

This supplement connects the integer comparison of an amount and a limit to the two derived commitments verified by Horizon. It provides a conditional derivation and a correlation protocol between JavaScript BigInt computation, a MATLAB/GNU Octave oracle and real Bulletproofs execution in Rust. Success conditions distinguish application rejection from cryptographic rejection. Finite tests, even when they agree, prove neither absence of defects nor the primitive's security. The numerical source is portable; the runtime actually executed must be read from each artifact. No MATLAB execution is inferred from an Octave result. The profile is not post-quantum.

## 1 Statement: arithmetic relation and policy

The examined profile is `horizon-range-v1`, in `labs/horizon/proof/src/lib.rs`. This supplement accompanies the manuscript `docs/whitepaper/whitepaper.tex` without replacing it. The campaign baseline is commit `5cf0ee2`; each run must identify its actual source revision and any local modifications.

Set $U = 2^{64} - 1$ and $N = 2^{32}$. The input domain is $\mathcal{D} = \{0, \dots, U\}^2$. The numerical decision is

$$P(a, c) \iff 1 \le a \le N \quad \wedge \quad a \le c \quad \wedge \quad c - a < N. \tag{1}$$

This relation is not simply $a \le c$. For example $(0, 0)$, $(N + 1, N + 1)$ and $(1, N + 1)$ satisfy the comparison but not $P$. The limit $c$ is a `u64` and may reach $2^{33} - 1$; imposing $c \le 2^{32} - 1$ would change the relation.

**Policy boundary.** Cryptographic acceptance also requires signatures, the case join, EUR currency, exponent 2, pinned keys and roles, timestamps, the independent challenge and the permitted mode. Authorizing an effect further requires revocation, epoch, nonce and business operation checks in the controller. The numerical oracle receives none of these elements. It therefore cannot grant business authorization. The limit remains per-case: two amounts of 60 under a limit of 100 do not establish a cumulative budget of 100; a deductible, unit conversion and payment rule would be additional relations.

## 2 From the integer witness to commitments

Let $\mathbb{G}$ be the Ristretto group of prime order

$$q = 2^{252} + 27742317777372353535851937790883648493.$$

Canonical encodings and the group abstraction are described in RFC 9496 [3]. Bases $G, H$ are the fixed public parameters of `PedersenGens::default()`, never chosen by the caller. Two authorities construct the signed commitments

$$A = aG + r_a H, \qquad C = cG + r_c H, \qquad r_a, r_c \in \mathbb{Z}_q.$$

The verifier derives the points without accepting ones separately proposed by the prover:

$$X = A - G, \qquad D = C - A. \tag{2}$$

The witness for the aggregated range proof is

$$x = a - 1, \quad d = c - a, \quad r_X = r_a, \quad r_D = r_c - r_a \pmod{q}.$$

Horizon uses Bulletproofs 5.0.0 `RangeProof` with two values and 32 bits [2]. This path is a specialized range proof; it does not compile the entire policy into a universal R1CS circuit. The word circuit in the campaign refers to this cryptographic relation.

**Lemma 1** (Integer equivalence)**.** *For $(a, c) \in \mathcal{D}$, $P(a, c)$ is equivalent to the existence of integers $x, d \in [0, N-1]$ such that $a = x + 1$ and $c = x + 1 + d$.*

*Proof.* If $P$ is true, $x = a - 1$ and $d = c - a$ satisfy the bounds. Conversely, $a = x + 1 \in [1, N]$ and $c - a = d \in [0, N - 1]$. These identities also give $c \leq 2N - 1$, without any addition that could exceed $U$. □

**Lemma 2** (Arithmetic completeness)**.** *If $P(a, c)$ is true, the preceding openings produce exactly the points in (2).*

*Proof.* $xG + r_X H = (a - 1)G + r_a H = A - G$. Likewise, $dG + r_D H = (c - a)G + (r_c - r_a)H = C - A$. Both values are in the required ranges; cryptographic completeness remains that of the primitive. □

## 3 Conditional soundness: knowledge and binding

**Assumption 3** (Issuance, primitive and implementation)**.** *The authorities have constructed or checked the openings of $A$ and $C$ from values in the `u64` domain, with appropriate metadata. The discrete logarithm between $G$ and $H$ is unknown to an efficient adversary. The primitive provides an argument of knowledge of openings within the ranges under its security model. Encodings, parameters, transcript, RNG and verification faithfully implement this primitive.*

The Bulletproofs paper and its corrected revision are the reference, with no reduction or software audit reinvented here [1]. Mere mathematical existence of openings would be insufficient: since $H$ generates $\mathbb{G}$, every point admits an opening to any scalar value with an appropriate mask. Moving to an attested value requires extractable knowledge and computational binding.

**Lemma 4** (Two incompatible openings)**.** *If $vG + rH = v'G + r'H$ and $v \not\equiv v' \pmod{q}$, then*

$$\log_G H = (v - v')(r' - r)^{-1} \pmod{q}.$$

*Proof.* Equality implies $(v - v')G = (r' - r)H$. If $r' = r$, the values would be congruent; otherwise $r' - r$ is invertible in the prime-order field. The formula gives the discrete relation assumed to be unknown. □

**Proposition 5** (Binding to attested integers without wraparound)**.** *Assume the preceding assumptions. An accepted proof from which $(x, d, r_X, r_D)$ is extracted with $0 \leq x, d < N$ imposes $P(a, c)$ on the attested values, except upon extraction failure or a binding break.*

*Proof.* The derived points give an opening of $A$ to $a' = x + 1$ with mask $r_X$, and of $C$ to $c' = x + 1 + d$ with mask $r_X + r_D$. If these values differ modulo $q$ from the attested openings, the preceding lemma yields a binding break. Otherwise $a \equiv a'$ and $c \equiv c' \pmod{q}$. Now $0 \leq a, c \leq U < q$ and $0 \leq a', c' \leq 2^{33} - 1 < q$. Two representatives in $[0, q)$ congruent modulo $q$ are equal. Hence $a = a'$ and $c = c'$, and integer equivalence concludes the proof. □

The bound $U < q$ is essential: an authority attesting an integer without a domain could sign a value $a' + q$ with the same scalar. Likewise, blindly signing a supplied point does not guarantee that the point represents the amount from the business system. Agreement among tests does not eliminate these risks.

Authority signatures are verified outside the range proof. Mode A includes the full challenge in the transcript; B reuses a historical proof and signs a new presentation with the holder key. B does not demonstrate fresh knowledge of the witness. Composition also assumes signature unforgeability, collision resistance of the digest, domain separation and full challenge comparison. No numerical security probability is inferred from tests.

## 4 MATLAB/Octave oracle: exact integers

The input to `math/circuit/circuit_oracle.m` is a JSON array of 1 to 10 000 objects containing exactly `id`, `category`, `amount` and `cap`. Numerical values are canonical decimal strings, never JSON numbers implicitly converted to `double`. The `jsondecode` API documents this conversion and field-name normalization [6]. Shape, key, duplicate and encoding checks therefore supplement standard decoding. An `expected` field is rejected. The oracle reads no expected answer from the corpus.

**Lemma 6** (Decimal conversion without saturation)**.** *Set* $Q = \lfloor U/10 \rfloor = 1844674407370955161$ *and* $R = 5$*. For a prefix* $v \in [0, U]$ *and digit* $b \in [0, 9]$*, the operation* $10v + b$ *is representable if and only if* $v < Q$*, or* $v = Q$ *and* $b \leq R$*.*

*Proof.* $U = 10Q + R$. If $v < Q$, $10v + b \leq 10Q - 1 \leq U$. If $v = Q$, the condition is exactly $b \leq R$. If $v > Q$, $10v + b \geq 10Q + 10 > U$. Induction over digits justifies checking before multiplication and addition. $Q$ is constructed from small, exactly representable blocks without converting a large floating-point literal. □

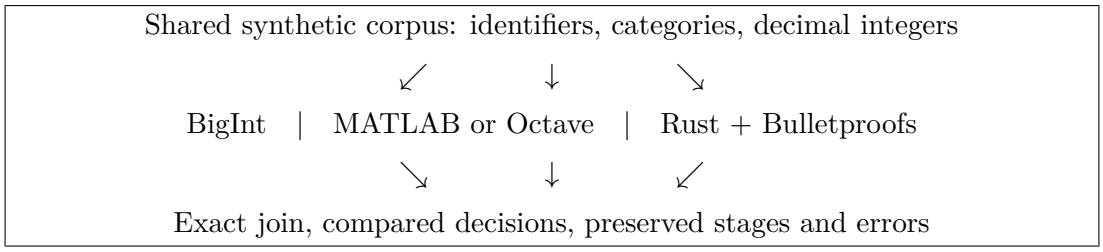
Computing $x$ is preceded by $a \geq 1$, and computing $d$ by $c \geq a$. This guard is essential: integer operations may saturate at type limits in MATLAB and Octave [5, 4]. A negative difference is returned as an empty string, not zero; other differences are returned exactly, including for a rejected case.

Decimal formatting also avoids integer division at the type boundary. For a value greater than $2^{32} - 1$, binary division by 10 keeps a remainder in $[0, 9]$; before reduction, this remainder is at most 19. The quotient is constructed bit by bit. For a value at most $2^{32} - 1$, the short path uses `floor(double(v)/10)`: the integer is represented exactly and the quotient's rounding error is far below 0.1, the minimum distance from a noninteger quotient to the next integer. No large `uint64` is sent through this path.

**Development observation.** On GNU Octave 7.3.0 in the recorded local image, an initial version using `idivide` at the type maximum did not pass formatting tests: $2^{64} - 1$ was rendered with an incorrect decimal ending. These tests failed before any positive result was published. The supplied source does not use this division. This observation is a local runtime result, not a conclusion about all MATLAB/Octave versions.

## 5 Protocol for correlating the three paths

For each vector $i$, let $J_i$ denote the BigInt decision, $M_i$ that of the numerical runtime actually executed, and $R_i$ the appropriate Rust result. The corpus and every result must carry unique identifiers and be linked by their hashes. Equality of identifier sets, values and counts is checked *before* equality of decisions. An omission, duplicate, process error or unreadable output never counts as a successful business rejection.




MATLAB and Octave running the same source are not two independent implementations of the predicate. Agreeing paths may share a specification error. The algebraic proof and business counterexamples therefore remain necessary.

| Test class | Success condition and meaning of the result |
|---|---|
| Valid arithmetic case | $J_i = M_i =$ true; A and B proofs generated, then accepted by a separate verifier process against pinned expectations. |
| Invalid arithmetic case | $J_i = M_i =$ false; the Rust wrapper may reject before generation. This rejection does not test the range verifier. |
| Bypassing the prover guard | Use the library API to construct an out-of-range attempt; observe generation failure or cryptographic rejection separately. Do not count the absence of a proof as a rejected proof. |
| Modified proof | Preserve a structurally valid encoding and, when required, re-sign the fixture bundle. Rejection must reach the claimed cryptographic stage rather than stopping at an invalid outer signature. |
| Modified context or policy | Authorization rejection distinct from arithmetic failure; unchanged effect count in the relevant application tests. |

The campaign verifier receives neither masks nor the holder's private key. Corpus amounts are synthetic and public to compare oracles; this is not an anonymity test. Proofs, metadata and commitments remain linkable. A separate process demonstrates a tested input boundary, not hardware isolation or an external audit.

## 6 Coverage, artifacts and interpretation

Sizes 128, 1 024 and 4 096 are campaign parameters, not assumed results. A report must state cases actually completed, their categories, both arithmetic verdicts, the exact Rust rejection point and proofs actually verified. Boundaries include 0, 1, $N-1$, $N$, $N+1$, $2N-1$, $2N$, $2^{53} \pm 1$, $U$ and the relations $c < a$, $c = a$, $c - a = N - 1$, $c - a = N$. Parsing tests include $U + 1$, leading zeros, signs, JSON numbers, extra fields and duplicate identifiers.

The local self-test run under Octave 7.3.0 on 17 October 2026 checked 11 boundary cases, rejected 36 invalid inputs and refused a file overwrite. It traversed all 4 096 pairs in the square $[0, 63]^2$, of which 2 016 satisfy $P$. This self-test is numerical; it does not execute Rust. The status of a complete correlated campaign is determined by its artifacts, not by these numbers or by compilation of this document.

Planned public files include `vectors.json`, `octave.json` or `matlab.json`, `prover-results.json`, `verifier-results.json`, `correlation.json`, compressed public verification requests, an HTML report and its figures. They are placed in a new run directory; no earlier archive is overwritten. The `runtime`, `runtime_version` and `executed_in_matlab` fields distinguish the engines actually used. A GitHub Actions run does not replace this attribution; a configured action or failed job is not a successful MATLAB execution.

Figures must derive from available results and include counts, units and errors. A panel without measurements is marked as not measured. Local latencies establish neither an SLA nor superiority over Vega: relations, issuance contracts and conditions differ. No GPU result is

inferred from this CPU test. Increasing corpus sizes do not measure cryptographic soundness probability or demonstrate exhaustive coverage of domain $\mathcal{D}$.

The conditional proofs here are written and reviewed without a formal proof assistant. Knowledge, binding and signature assumptions remain inherited; exactly-once remote authorization, source truth, side channels and banking qualification remain out of scope. A SHA-256 manifest binds bytes under a trusted reference; its local date is not a third-party timestamp and its content does not prove the semantics of a claim.

## References


[1] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Report 2017/1066, 2017. URL https://eprint.iacr.org/2017/1066. Revised 2022-04-14, correcting the Fiat–Shamir section. Consulted 2026-10-17.

[2] Dalek Cryptography. Bulletproofs 5.0.0: RangeProof API. URL https://docs.rs/bulletproofs/5.0.0/bulletproofs/struct.RangeProof.html. Pinned implementation documentation; consulted 2026-10-17.

[3] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, Informational, 2023. URL https://www.rfc-editor.org/rfc/rfc9496.html. Consulted 2026-10-17.

[4] GNU Octave. GNU Octave 7.3.0: Integer arithmetic. URL https://docs.octave.org/v7.3.0/Integer-Arithmetic.html. Version-specific primary manual; consulted 2026-10-17.

[5] The MathWorks, Inc. Integers, n.d. URL https://www.mathworks.com/help/matlab/matlab_prog/integers.html. Integer types, arithmetic and overflow; consulted 2026-10-17.

[6] The MathWorks, Inc. jsondecode: Decode JSON-formatted text, n.d. URL https://www.mathworks.com/help/matlab/ref/jsondecode.html. Primary API documentation; consulted 2026-10-17. Documentation is not evidence of local MATLAB execution.